

Mulberry for macOS v1 Architecture

Native Swift Client, Canvas Sync, Overlay Windowing, and Release Engineering

Mulberry Engineering

May 2026

Abstract

This document specifies the v1 architecture for Mulberry's native macOS client. It intentionally removes architectural ambiguity for implementers: platform APIs, module boundaries, persistence shape, sync behavior, canvas operation handling, windowing behavior, testing obligations, packaging, and backend changes are fixed here. The app is a first-class Mulberry client, not a viewer. It signs in with Google, joins the same pair session as Android, applies and emits the same ordered canvas operations, and adds macOS-native presence through a passive desktop overlay and temporary Quick Draw mode.

1 System Overview

The macOS client is a native Swift application with three presentation surfaces:

1. a status bar menu that owns ambient commands and state;
2. a full SwiftUI app window for sign-in, canvas editing, settings, sticker browsing, troubleshooting, and account management;
3. an AppKit overlay panel for passive desktop consumption and Quick Draw.

The client communicates with the existing Fastify backend through REST and WebSocket APIs. The backend remains the authority for authentication, pair membership, operation ordering, server revisions, recovery, profile state, invites, sticker catalog, wallpaper catalog where reused, reactions, and streaks.

2 Repository and Target Layout

The production target must live under `apps/macos`. The overlay proof of concept is intentionally kept outside the monorepo app slot at `prototypes/macos-overlay-poc`; production code must be organized as an app bundle target rather than a single-file executable.

2.1 Required Package Layout

```
apps/macos/  
  MulberryMac.xcodeproj or Package.swift  
  Sources/  
    MulberryMacApp/  
    AppShell/  
    Auth/  
    Networking/  
    Sync/  
    CanvasCore/  
    CanvasRendering/
```

- Persistence/
- Overlay/
- QuickDraw/
- Stickers/
- Reactions/
- Notifications/
- Settings/
- Diagnostics/
- Tests/
 - CanvasCoreTests/
 - SyncTests/
 - PersistenceTests/
 - OverlayTests/
 - NetworkingTests/
- Resources/
 - Assets.xcassets
 - Fonts/

The app may start as a Swift Package for speed, but the release candidate must produce a signed app bundle and DMG. If Swift Package Manager blocks app bundle capabilities, migrate to an Xcode project while preserving the module folders.

3 Module Responsibilities

Module	Responsibilities
MulberryMacApp	App entry point, dependency graph, app lifecycle, scene/window registration.
AppShell	Status bar item, menu model, full app window routing, activation policy, single-instance behavior.
Auth	Google sign-in, session refresh, sign-out, Keychain token storage.
Networking	REST client, request signing, retry policy, DTO decoding, back-end error mapping.
Sync	WebSocket client, HELLO/READY/ACK handling, reconnect, outbound queue, REST recovery.
CanvasCore	Platform-independent Swift canvas domain models and operation reducer.
CanvasRendering	Core Graphics/SwiftUI rendering for strokes, text, stickers, active strokes, and snapshots.
Persistence	SQLite-backed local store, migrations, operation ledger, snapshot cache, settings store.
Overlay	Passive overlay panel, display selection, frame persistence, window levels, Spaces behavior.
QuickDraw	Global hotkey, raised overlay mode, tool palette, Quick Draw lifecycle, fullscreen guard.
Stickers	Sticker pack catalog client, asset cache, signed URL loading, render metadata.

Reactions	Send reaction, receive/lease reaction playback if supported, local feedback.
Notifications	User-visible notifications, permission state, notification routing, login item reconnect messaging.
Settings	Preferences for overlay, login item, notifications, account, diagnostics.
Diagnostics	Logs, sync debug state, recovery actions, exportable diagnostic bundle.

4 Platform Decisions

4.1 Language and UI

Use Swift 6 or newer and SwiftUI for primary UI. Use AppKit for status bar, overlay panels, global hotkey integration, window levels, and any drawing/input behavior where SwiftUI window abstractions are insufficient. Do not use Electron, Tauri, Catalyst, or a web view shell for v1.

4.2 Minimum macOS Version

Target macOS 14 or newer. macOS 15 or newer is acceptable if a concrete implementation dependency justifies it, but the default floor is macOS 14 to keep the first release reasonably reachable while using modern SwiftUI, Service Management login item APIs, and AppKit behavior.

4.3 Global Hotkey

Use Carbon’s registered hotkey API for Command-Control-M in v1. Do not use an `NSEvent` global monitor as the primary hotkey mechanism because it is not a reliable universal shortcut primitive and can be gated by permissions. Detect hotkey registration failure and display a Settings warning.

4.4 Login Item

Use `SMAccessService` for login item registration. The default should be opt-in during onboarding with clear copy. Once enabled, launch at login into accessory mode, restore overlay, and begin sync.

5 Windowing Architecture

5.1 Passive Overlay

Implement the passive overlay as an AppKit `NSPanel` subclass. It must be borderless, transparent, shadowless, and click-through.

Required configuration:

- `styleMask = [.borderless]`
- `backgroundColor = .clear`
- `isOpaque = false`
- `hasShadow = false`

- `ignoresMouseEvents = true`
- `collectionBehavior` includes `.canJoinAllSpaces`, `.stationary`, and `.ignoresCycle`
- `level = NSWindow.Level(rawValue: NSWindow.Level.normal.rawValue - 1)`

Do not promise exact layering against desktop icons, Stage Manager thumbnails, desktop widgets, or Finder private windows. The contract is below normal application windows and above the wallpaper where macOS permits.

5.2 Display Selection

Only one overlay window exists. It is pinned to one selected display. Persist:

- display identifier where available;
- last known display frame;
- overlay frame in display-relative coordinates;
- whether overlay is visible.

On display changes:

1. try to resolve the stored display identifier;
2. if unavailable, choose the main display;
3. clamp the overlay frame inside the visible frame;
4. log a diagnostic event if the display changed.

5.3 Quick Draw Mode

Quick Draw reuses the same overlay panel and adds a tool palette panel. It is not a persistent always-on-top mode.

On entry:

- verify not currently inside another app's fullscreen Space;
- activate Mulberry;
- set overlay level to `.floating`;
- set `ignoresMouseEvents = false`;
- show the tool palette adjacent to the overlay;
- make the overlay key;
- begin a Quick Draw session in app state.

On exit:

- finish or cancel the active stroke based on pointer state;
- flush queued operations;
- hide tool palette;
- lower overlay to passive level;
- set `ignoresMouseEvents = true`;
- end the Quick Draw session.

Quick Draw must expose full drawing-canvas feature parity. It cannot be a reduced brush-only surface. The tool palette and any secondary popovers must support brush, color, width, eraser, undo, redo, clear, text creation and editing, sticker insertion, element selection, transform affordances, and Done. If the compact palette cannot fit every control at once, use segmented tool groups and popovers; do not redirect the user to the full app for a canvas operation that the full app supports.

6 Authentication Architecture

Use Google Sign-In only for v1. The preferred implementation is a browser-based OAuth flow using `ASWebAuthenticationSession` with the same server-side Google ID token verification model as Android. If the Google SDK is selected, it must still produce an ID token acceptable to the existing backend.

6.1 Token Storage

Store refresh tokens and long-lived credentials in Keychain. Store access token expiry and non-secret bootstrap state in SQLite or app settings. Never store access tokens in `UserDefaults`. On sign-out, delete Keychain entries, local pending operations, local snapshots, cached partner state, and notification tokens.

7 Networking

7.1 REST Client

Use `URLSession`. Define typed request/response DTOs matching the backend. All authenticated REST calls pass `Authorization: Bearer <accessToken>`. On 401, refresh once and retry. If refresh fails, transition to signed-out and stop sync.

Required endpoints:

- `POST /auth/google`
- `POST /auth/refresh`
- `POST /auth/logout`
- `GET /bootstrap`
- `GET /streak`
- `PUT /me/presence-surfaces/:surfaceType`
- `PUT /me/display-name`
- `PUT /me/profile`
- `PUT /me/partner-profile`
- `POST /reactions/send`
- `GET /canvas/ops`
- `POST /canvas/ops/batch`
- `GET /canvas/snapshot`
- `GET /stickers/packs`
- `GET /stickers/packs/:packKey`
- `GET /stickers/assets/url`

7.2 WebSocket Client

Use `URLSessionWebSocketTask`. The client sends `HELLO` after opening:

```
{
  "type": "HELLO",
  "accessToken": "...",
  "pairSessionId": "...",
  "lastAppliedServerRevision": 123
}
```

The client must handle `ready`, `ack`, `batch ack`, `server operation`, `server operation batch`, `flow control`, `error`, and `pong` messages. Prefer batches for stroke append throughput.

8 Persistence Architecture

Use SQLite through GRDB or a thin SQLite wrapper. GRDB is preferred for migrations, typed queries, and testability. Do not use Core Data for v1 because the operation ledger is append-heavy, revision-ordered, and benefits from explicit SQL constraints.

8.1 Tables

Table	Purpose
<code>app_metadata</code>	schema version, installation ID, last migration time.
<code>session_state</code>	non-secret auth metadata, user ID, email, display name, pair session ID.
<code>bootstrap_cache</code>	latest bootstrap payload, partner state, streak summary.
<code>presence_surfaces</code>	latest local view of own and partner Android wallpaper and macOS overlay presence.
<code>canvas_operations</code>	accepted server operations by pair session and revision.
<code>pending_operations</code>	local operations not yet accepted by server.
<code>canvas_snapshot</code>	latest server or locally materialized snapshot.
<code>sticker_packs</code>	sticker pack metadata keyed by pack key and version.
<code>sticker_assets</code>	local asset paths, signed URL expiry, cache status.
<code>overlay_settings</code>	visibility, selected display, frame, passive opacity policy.
<code>app_settings</code>	login item, notifications, hotkey preference, diagnostics.

8.2 Operation Constraints

- `canvas_operations` has a unique index on `(pairSessionId, serverRevision)`.
- `canvas_operations` has a unique index on `clientOperationId`.
- `pending_operations` has a unique index on `clientOperationId`.
- Reapplying an accepted operation with an already-known revision is a no-op.
- Reapplying an accepted operation with an already-known client operation ID is a no-op if payload matches; payload mismatch is a diagnostic error.

9 Canvas Core

CanvasCore is the deterministic reducer for canvas state. It must not depend on SwiftUI, AppKit windows, URLSession, Keychain, or SQLite. It may depend on Foundation and CoreGraphics.

9.1 Domain Types

Required Swift models:

- **CanvasState**: committed strokes, elements, active local stroke, remote active strokes, revision, snapshot metadata.
- **CanvasStroke**: ID, color ARGB, width, points, createdAt.
- **CanvasPoint**: normalized x/y Float values in the canonical canvas coordinate system.
- **CanvasElement**: enum with text and sticker cases.
- **CanvasTextElement**: ID, text, createdAt, center, rotation, scale, boxWidth, colorArgb, backgroundPillEnabled, font, alignment.
- **CanvasStickerElement**: ID, createdAt, center, rotation, scale, packKey, packVersion, stickerId.
- **CanvasOperation**: local and server forms with clientId, type, strokeId, payload, clientCreatedAt, local date, serverRevision.

9.2 Operation Reducer

The reducer implements the existing operation names exactly:

- ADD_STROKE
- APPEND_POINTS
- FINISH_STROKE
- DELETE_STROKE
- CLEAR_CANVAS
- ADD_TEXT_ELEMENT
- UPDATE_TEXT_ELEMENT
- DELETE_TEXT_ELEMENT
- ADD_STICKER_ELEMENT
- UPDATE_STICKER_ELEMENT
- DELETE_STICKER_ELEMENT

All coordinates stored and transmitted by macOS are normalized to the canonical canvas surface, not raw window pixels. Rendering maps normalized coordinates to the current view rectangle. This keeps Android, overlay, and full app surfaces interoperable.

10 Rendering Architecture

Use Core Graphics for deterministic drawing and SwiftUI wrappers for presentation. Passive overlay and full app must share the same renderer. The renderer receives a **CanvasRenderInput** with state, viewport, scale, asset resolver, and render mode.

10.1 Render Order

The render order is fixed:

1. optional local background if the surface needs one;
2. committed strokes in operation order;
3. active remote strokes;
4. active local stroke;
5. text and sticker elements in element order;
6. transient reaction overlay if active;
7. selection handles only on interactive surfaces.

Passive overlay must render transparent background and full-opacity content. Quick Draw and the full app may render an editing surface background.

11 Sync State Machine

State	Meaning
SignedOut	No session. Sync stopped.
Bootstrapping	Authenticated, fetching bootstrap and local state.
Disconnected	Authenticated and paired, no active WebSocket.
Connecting	Opening WebSocket and sending HELLO.
Recovering	Fetching missed operations or snapshot fallback.
Connected	WebSocket ready; live operations flow.
Backoff	Connection failed; waiting before retry.
Error	User-visible issue requiring action.

Reconnect backoff starts at 1 second, doubles up to 60 seconds, and resets after a stable connection of 30 seconds. A network reachability change triggers an immediate retry if not signed out.

12 Backend Changes

12.1 Device Platform

Extend device platform support from Android-only to include macOS. The server type should accept `ANDROID`, `MACOS`, and future Apple platforms without needing another migration. Add app environment, app version, device name, and optional push token type.

12.2 Presence Surfaces

Add a platform-aware presence-surface model rather than adding user-level `overlayConfigured` columns. Presence is per user, pair session, device instance, and surface type. Required surface types for v1 are:

- `ANDROID_WALLPAPER`
- `MACOS_OVERLAY`

The table should support at least:

- user ID;
- pair session ID;
- device instance ID;
- surface type;
- configured state;
- enabled or visible state;
- `canSeeLatestDrawings`;
- `hasEverBeenAbleToSee`;
- details JSON for surface-specific facts such as Android home/lock selection or macOS display/overlay visibility;
- updated timestamp.

The backend computes aggregate partner visibility from these rows. A partner can see latest drawings if any active configured surface can see latest drawings. Per-surface status remains available for settings, diagnostics, and targeted troubleshooting.

The existing Android wallpaper contract must remain backward compatible while the new presence table is introduced:

- keep `user_wallpaper_status`;
- keep Android PUT `/me/wallpaper-status`;
- keep bootstrap `partnerWallpaperStatus`;
- keep the `PARTNER_VISIBILITY_CHANGED` push payload and timing.

Android wallpaper status writes should also update the `ANDROID_WALLPAPER` presence surface so the aggregate model stays current.

macOS reports `MACOS_OVERLAY` using a stable installation/device instance ID. Phase 5 may report configured and enabled metadata once the user has authenticated and bootstrap has completed, but it must not report `canSeeLatestDrawings = true` until the app can render the latest canvas through Phase 6 and stay current through Phase 7 sync/recovery.

12.3 Push and Notifications

V1 can ship without APNs silent push if the login item keeps the app running. However, the backend schema should not block APNs later. Store push provider and token separately:

- platform: `ANDROID` or `MACOS`;
- pushProvider: `FCM`, `APNS`, or `NONE`;
- token: nullable;
- appEnvironment: `dev` or `prod`;
- deviceInstanceId;
- revokedAt.

Add a surface-aware notification payload, `PARTNER_PRESENCE_CHANGED`, for future clients that understand cross-platform presence. Do not replace the Android `PARTNER_VISIBILITY_CHANGED` payload in v1; Android notifications for wallpaper visibility must continue to fire with the legacy fields and timing.

12.4 Canvas Protocol

No new canvas operation types are required for v1. The only backend sync requirement is to preserve support for multiple active connections for the same user in the same pair session and to keep operation acceptance idempotent.

13 Testing Strategy

13.1 Unit Tests

Required unit suites:

- Canvas operation reducer for every operation type.
- JSON encode/decode compatibility for WebSocket and REST DTOs.
- SQLite migrations and constraint behavior.
- Backend presence-surface aggregation and legacy Android wallpaper compatibility.
- Outbound queue idempotency and retry ordering.
- Display selection and frame clamping logic.
- Hotkey registration success and failure wrapper behavior.

13.2 Integration Tests

Required integration suites:

- local backend sync from Mac to Android-compatible protocol fixtures;
- recovery after dropped WebSocket;
- app restart with pending outbound operations;
- sticker asset cache invalidation;
- sign-in refresh and sign-out cleanup.

13.3 Manual QA Matrix

Manual QA must include:

- macOS 14 and latest macOS, or macOS 15 and latest macOS if the minimum is raised;
- one display and two displays;
- all-Spaces overlay behavior;
- Mission Control and Stage Manager if enabled;
- fullscreen app Quick Draw prompt;
- sleep/wake;
- network offline/online;
- login restart;
- hotkey conflict;
- Android and Mac drawing simultaneously.

14 Release Engineering

Direct DMG distribution is v1. The release pipeline must produce:

- signed app bundle;
- notarized DMG;
- versioned build number;
- release notes;
- crash/log symbol retention;
- rollback path to previous DMG.

Do not require an Apple Developer Program account for local development. Release signing and notarization can be added when credentials exist.

15 Security and Privacy

- Store secrets only in Keychain.
- Redact access tokens, refresh tokens, Google ID tokens, signed sticker URLs, emails, and canvas text content from logs.
- Diagnostics export must omit canvas content by default.
- Notifications must not display sensitive canvas text unless the user opts in.
- Overlay content is visible on the user’s desktop; onboarding must explain this plainly.

16 Implementation Invariants

The following decisions are fixed for v1:

1. Native Swift/SwiftUI plus AppKit, not Electron/Tauri.
2. Google Sign-In only.
3. Pairing remains user-to-user.
4. Mac devices participate equally with Android devices.
5. Passive overlay is below normal windows and click-through.
6. Quick Draw temporarily floats and takes focus.
7. Quick Draw is not supported over another app’s fullscreen Space.
8. Canvas sync uses the existing operation protocol.
9. Coordinates are normalized, not pixel-bound.
10. SQLite is the local source of truth for canvas ledger and cache; Keychain is the source of truth for secrets.
11. Partner visibility is surface-aware: aggregate visibility is true when any configured active surface can show latest drawings, while Android wallpaper compatibility remains intact.